

Software Architecture Design Methods

Lecturer: Ngo Huy Bien
Software Engineering Department
Faculty of Information Technology
VNUHCM - University of Science
Ho Chi Minh City, Vietnam
nhbien@fit.hcmus.edu.vn

Objectives

- To apply *systematic methods* to create a software architecture.



Contents

- I. The Attribute-Driven Design (ADD) Method.
- II. Viewpoints and Perspectives.

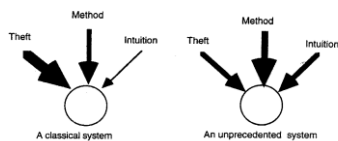


References

1. Philippe Kruchten (1995). Mommy, Where Do Software Architectures Come From.
2. Humberto Cervantes and Rick Kazman (2016). Designing Software Architectures - A Practical Approach. Addison-Wesley Professional.
3. Christine Hofmeister et al. (2007). A General Model of Software Architecture Design Derived From Five Industrial Approaches.
4. Nick Rozanski and Eoin Woods (2012). Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives. Addison-Wesley Professional.
5. Daniel Perovich et al. (2009). Model-Driven Approach to Software Architecture Design.
6. Jan Bosch and Peter Molin (1999). Software Architecture Design Evaluation and Transformation.



Architecture Sources [1]



The contribution of the 3 sources can have different profiles; for instance:

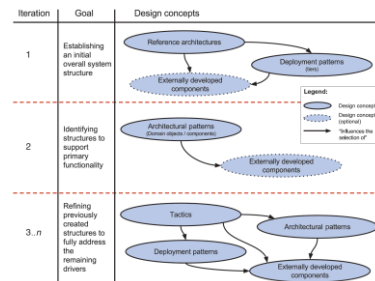
- Classical: 80% theft, 19% method, 1% intuition
- Unprecedented: 30% theft, 50% method, 20% intuition



The Attribute-Driven Design (ADD) Method

- The **attribute-driven design (ADD) method** can be used for creating
 - the design of a **greenfield** system for a **mature** (i.e., well-known) domain.
 - the design of a **greenfield** system for a domain that is **novel** (i.e., a domain that has a less established infrastructure and knowledge base)
 - the design for making changes to an **existing** system (**brownfield**).
- Each one of these categories involves a **different roadmap** in terms of the sequence of goals that you should perform across the design iterations.

Design of Greenfield Systems for Mature Domains [2]



Design of Greenfield Systems for Novel Domains

- Reference architectures** may not exist and there may be few, if any, **externally developed components**.
- General-purpose design concepts such as **tactics** and **patterns** can guide you.
- Your design goal will focus on **the creation of prototypes** so that you can explore possible solutions to the challenge that you are facing.



Design for an Existing System (Brownfield)

- You may also be making architectural changes to an existing system for the purpose of **refactoring**.
- When refactoring, you change the architecture of an existing system, **without** altering its functions, to reduce technical debt, to introduce technology updates, or to fix quality attribute problems.

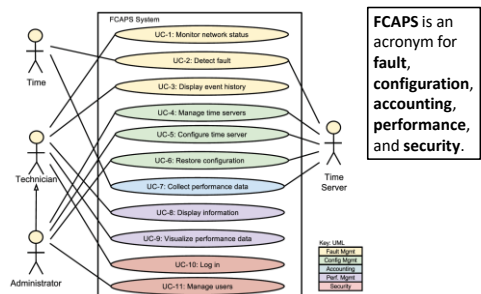


Round 1: Inputs

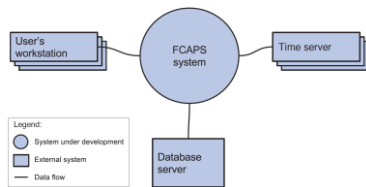
- Business Cases
- System Requirements
 - Use Case Model / User Stories / Story Map
 - Quality Attribute Scenarios
 - Constraints
- Architectural Concerns
 - Languages and technologies.
 - Initial system structure.
 - Allocating work for members of development team.



FCAPS System



Context Diagram



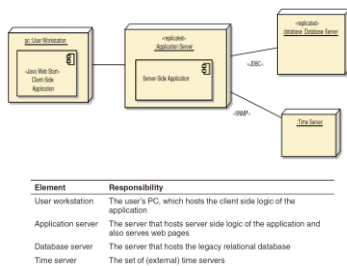
Overall System Structure

- Inputs:
 - Architectural styles
 - Reference architectures
 - Programming languages
 - Technologies
- Method: Decomposition



Deployment Diagram

- Decomposition
- Reference architectures
- Architectural styles
- Programming languages
- Technologies

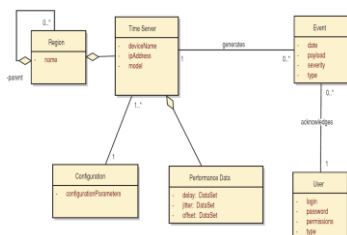


Round 2: Inputs

- Overall system structure.
- Deployment diagram.
- A primary workflow.

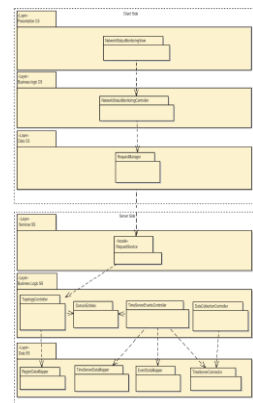


Initial Domain Model

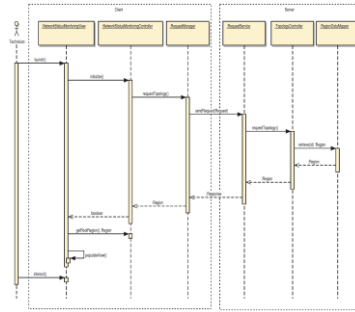


Package Diagram for the Primary Workflow

- Decomposition
- Reference architectures
 - Clean architecture
- Architectural styles
- Programming languages
- Technologies
- Domain-driven design



Sequence Diagram for the Primary Workflow



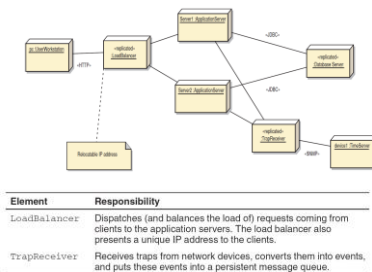
Round 3: Inputs

- Overall system structure
- Deployment diagram
- Package diagram
- Sequence diagrams
- A primary workflow
- A quality attribute scenarios associated with the primary workflow

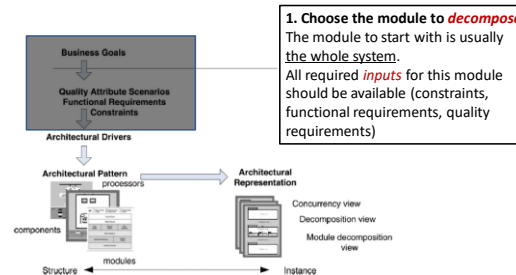


Refined Deployment Diagram

- Reference architectures
- Architectural styles
- Programming languages
- Technologies

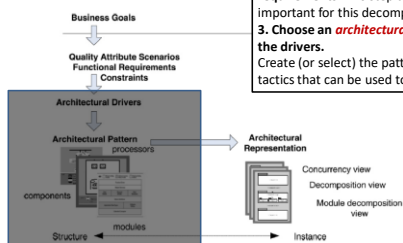


The Attribute-Driven Design (ADD) Method Steps (I) [3]



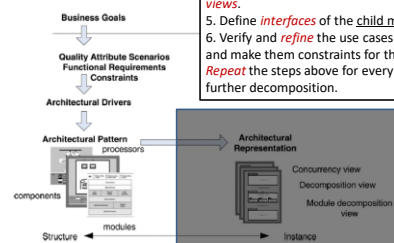
The Attribute-Driven Design (ADD) Method Steps (II)

- 2. Choose the *architectural drivers* from the set of concrete quality scenarios and functional requirements.** This step determines what is important for this decomposition.
- 3. Choose an *architectural pattern* that satisfies the drivers.**
Create (or select) the pattern based on the tactics that can be used to achieve the drivers.



The Attribute-Driven Design (ADD) Method Steps (III)

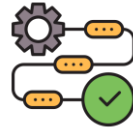
- 4. Instantiate *modules* and allocate functionality from use cases, and represent the results using *multiple views*.**
- 5. Define *interfaces* of the *child modules*.**
- 6. Verify and *refine* the use cases and quality scenarios and make them constraints for the child modules.**
Repeat the steps above for every module that needs further decomposition.





Viewpoints and Perspectives Method: Activities [4]

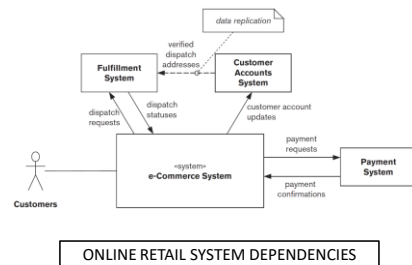
1. Agreeing on scope and context, constraints, and baseline architectural principles
2. Identifying and engaging stakeholders
3. Identifying and using architectural scenarios
4. Using architectural styles and patterns
5. Producing architectural models
6. Documenting the architecture
7. Validating the architecture



The Context Viewpoint

- The **Context view** of a system defines the relationships, dependencies, and interactions between the system and its environment—the people, systems, and external entities with which it interacts.
- **Concerns:** System scope and responsibilities.
- **Model:** Context diagram.

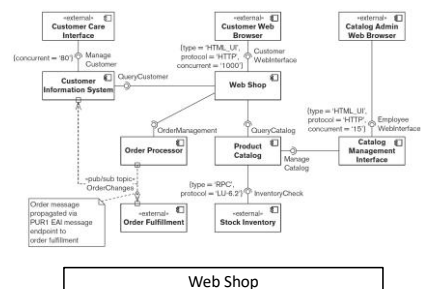
Context View Example



The Functional Viewpoint

- This view documents **the system's functional structure**—including the key functional elements, their responsibilities, the interfaces they expose, and the interactions between them.
- **Concerns:** Functional capabilities, External Interfaces, Internal Structure, Functional Design Philosophy.
- **Model:** Functional structure model (UML component diagrams, Boxes-and-lines diagrams).

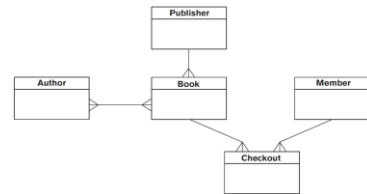
Functional View Example



The Information Viewpoint

- You use the **Information view** to answer, at an architectural level, questions about how your system will store, manipulate, manage, and distribute information.
- Concerns:** Information Structure and Content, Information Purpose and Usage, Information Ownership, Enterprise-Owned Information, Identifiers and Mappings, Volatility of Information Semantics, Information Storage Models, Information Flow, Information Consistency, Information Quality, Timeliness, Latency, and Age, Archiving and Information Retention.
- Model:** Static information structure models (entity-relationship diagrams, UML class diagrams, data flow diagrams, UML state diagrams).

Information View Example

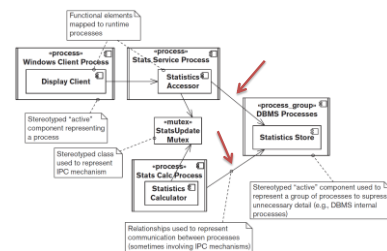


ENTITY-RELATIONSHIP DIAGRAM FOR LIBRARY

The Concurrency Viewpoint

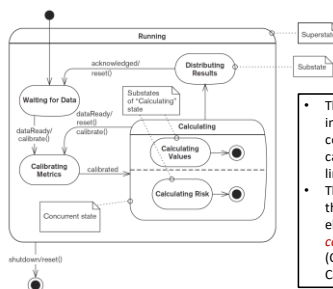
- The **Concurrency view** is used to describe the system's concurrency and state-related structure and constraints. This involves defining the parts of the system that **execute at the same time** and how this is controlled.
- To do this, you need to create a **process model** and a **state model**: The process model shows the planned process, thread, and inter-process communication structure; the state model describes the set of states that runtime elements can be in and the valid transitions between those states.
- Some information systems have little concurrency. Others, while exhibiting concurrent behavior, **use the facilities of underlying frameworks** and containers (e.g., application servers and databases) to **hide the concurrency model** in use.
- Many of today's information systems are inherently event-driven, reactive, concurrent systems.
- Concerns:** Task (Process or Thread) Structure, Inter-process Communication, State Management, Synchronization and Integrity, Supporting Scalability, Task Failure,
- Model:** System-Level Concurrency Models, State Models.

Concurrency View Example 1



The concurrent activity between the Statistics Accessor and Statistics Calculator components needs to be coordinated.

Concurrency View Example 2

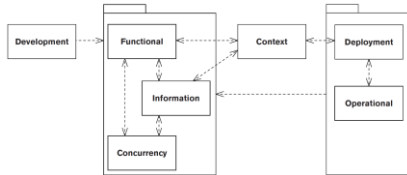


- The **Calculating state** is interesting because it is a concurrent state, as you can see from the dashed line that bisects it.
- This means that while in the **Calculating state**, the element is actually in two **concurrent substates** (Calculating Values and Calculating Risk).

The Development Viewpoint

- It is this view that addresses the specific concerns of the software **developers** and **testers**.
- The importance of this view depends on **the complexity of the system** being built, the expertise of the software developers, the maturity of **the technologies used**, and the familiarity that the whole team has with these technologies.
- Concerns:** Module Organization, Common Processing, Standardization of Design, Standardization of Testing, Instrumentation, Codeline Organization.
- Model:** Module Structure Models (UML package diagrams).

Dependencies Between Views



Core Perspectives

- Security.
- Performance and Scalability.
- Availability and Resilience.
- Evolution.



Perspective Details

- The perspective's *applicability to views*—that is, which of your views are most likely to be impacted by applying the perspective.
- The most important *concerns* the perspective addresses.
- A description of *activities for applying the perspective* to your architecture.
- The key *architectural tactics* to be considered as possible solutions when your architecture does not exhibit the required quality properties the perspective addresses.
- Some *problems and pitfalls* to be aware of and risk-reduction techniques for mitigating these.
- *Checklists* of things to consider when applying the perspective and when reviewing it, to help ensure correctness, completeness, and accuracy.



RUP's Architectural Design Activities [3]

- Architectural design activities in RUP *start with the following artifacts*, which may not be finalized and still evolve during the architectural design:
 - a vision document,
 - a use case model (functional requirements) and
 - supplementary specification (non-functional requirements, quality attributes).
- The three main groups of *activities* are:
 - Define a *candidate architecture*. This usually starts with a use-case analysis, focusing on the use cases that are deemed architecturally significant, and with any *reference architecture* the organization may reuse.
 - Perform Architectural Synthesis. Build an *architectural proof-of-concept*, and assess its viability, relative to functionality and to non-functional requirements.
 - Refine the Architecture. Identify *design elements* (classes, processes, etc.)

Application-Generic Knowledge: Patterns, Standards, Frameworks

- *Patterns*
 - Gang of Four
 - Buschmann and his Gang of Five.
- *Standards*
 - TOGAF (The Open Group Architectural Framework)
 - Zachman Framework
 - ISO 19439 (FDIS), etc.
- *Frameworks*
 - IEEE Std 1471
 - ISO/IEC 42010, etc.

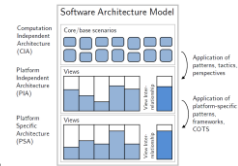


Model-Driven Approach [5]

- A *specialization of the ADD method*, using Model-Driven Engineering techniques, that systematizes and assists the Decision Making activity.

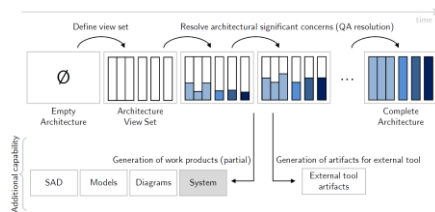
- The architecture *representation* is precisely stated.

- six architectural viewpoints, each addressing a cohesive set of architectural concerns: *Functional, Information, Concurrency, Development, Deployment, and Operational*.
- use *UML notation* for depicting models, and somehow complement the *language definitions* provided by the view type approach.
- structure architectural views in *three levels of abstraction*.



Model Transformations

- The method is understood as the successive application of *model transformations*, starting from an empty representation and ending with the complete architecture representation.



How to Create an Architecture? [6]

- Impose *architectural style*
- Impose *architectural pattern*
- Apply *design pattern*
- Convert NFRs to *functionality*
 - Exception handling is a well-known example that adds functionality to a component to increase the fault-tolerance of the component.
- *Distribute requirements*
 - The *divide-and-conquer principle*: a NFR at the system level is distributed to the subsystems or components that make up the system.

Thank You & See You Again

